



Vending Machine

SOFT2412: Agile Software Development Practices
Sprint 2 Sprint Report

Contents

1	Executive Summary	1
2	Scrum	2
2.1	Sprint Goals & Overview	2
2.2	User Ideas to Propose to Client	5
2.3	Mid-fi Diagrams	5
2.4	Product and Sprint Task Boards	7
2.5	Artifacts	7
2.5.1	Standups	7
2.5.2	Sprint Review	7
3	Agile Development Practices	9
3.1	Git	9
3.1.1	Branches	9
3.2	Testing	10
3.2.1	Incremental Test Case Design	10
3.2.2	Integrated Component Testing	10
3.2.3	Iterative Testing	10
3.2.4	Test Coverage	10
3.3	Gradle	12
3.3.1	Build Automation	12
3.3.2	OpenCSV	12
3.4	CI/CD Tools and Practices	13
4	Application Development	15
4.1	Group Collaboration	15
4.2	Application Design	15
A	UML diagram	16
B	Mid-fi diagrams	18
B.1	Transaction pages	18
B.2	Manage users page	22
C	Meeting Minutes	23

1 Executive Summary

In Sprint 2 of the project, development revolved around the connecting the various basic components built in Sprint 1. Specifically, the team attempted to complete the guest user and customer use case (as can be seen in the use case diagram in the appendix). This was due to the fact that the other use cases were codependent on completing the basic use cases in the customer/registered customer use case. For example, for it would be counter intuitive to implement a cashier being able to add or remove cash from the vending machine when customer transactions have not been implement. The focus for this sprint has been to build a robust application that can shown to the client. This way the client has a clear idea of how the final product will look like. Additionally, the team aimed to complete the extra requirements that the client had specified in the first client meeting. These include rotating and enlarging the tab labels to ensure better readability, and providing functionality allowing owner to promote existing users to different roles, such as promoting an existing seller to an owner.

2 Scrum

2.1 Sprint Goals & Overview

Sprint 2 Goal

To implement a basic shopping experience in the Vending Machine application, where a user can add items to a cart and successfully checkout using either cash or card.

At the start of the sprint the team documented all the objectives that needed to be met, and discussed how best they could be achieved. This sprint goals are listed below.

- During the client meeting at the end of the first sprint, the client tasked the team with implementing two additional features into the application.
 1. To change the orientation of the category tabs from vertical to horizontal and enlarge them for better visibility (see Figure 1).
 2. To implement a feature to allow the owner of the application to elevate and de-elevate the permissions/role of the user.

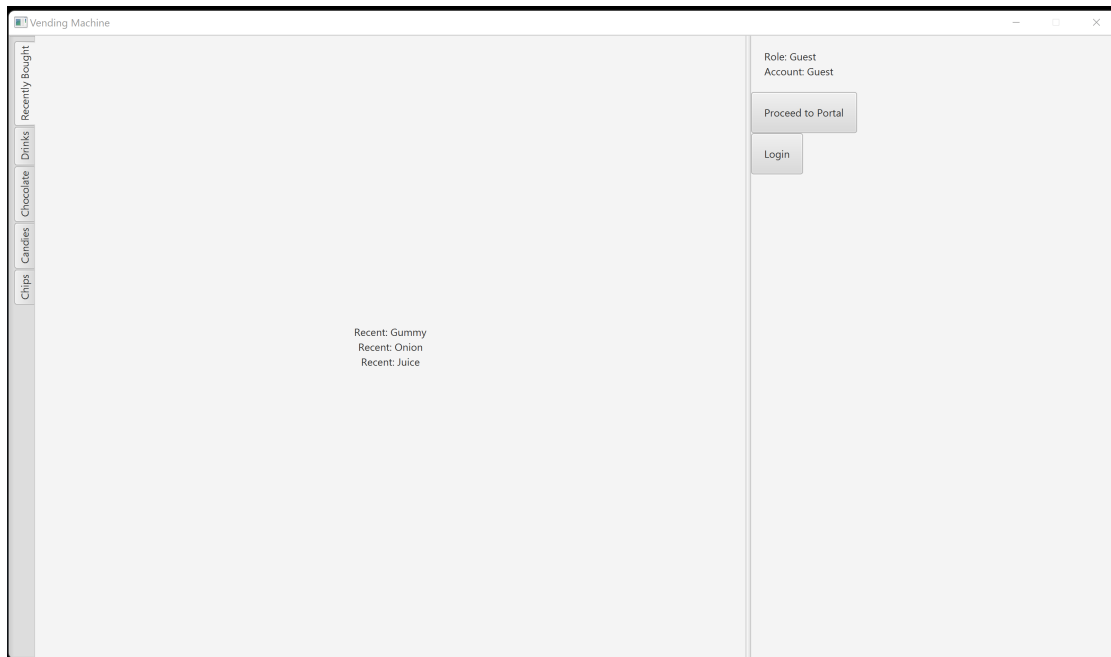
Implementing these features was a high-priority for the team in this sprint.

- Implement the “Customer” use case as seen in the use case diagram (Figure
- Begin implementing transaction features into the app: a cart, a checkout page, the option to pay with cash or card etc.

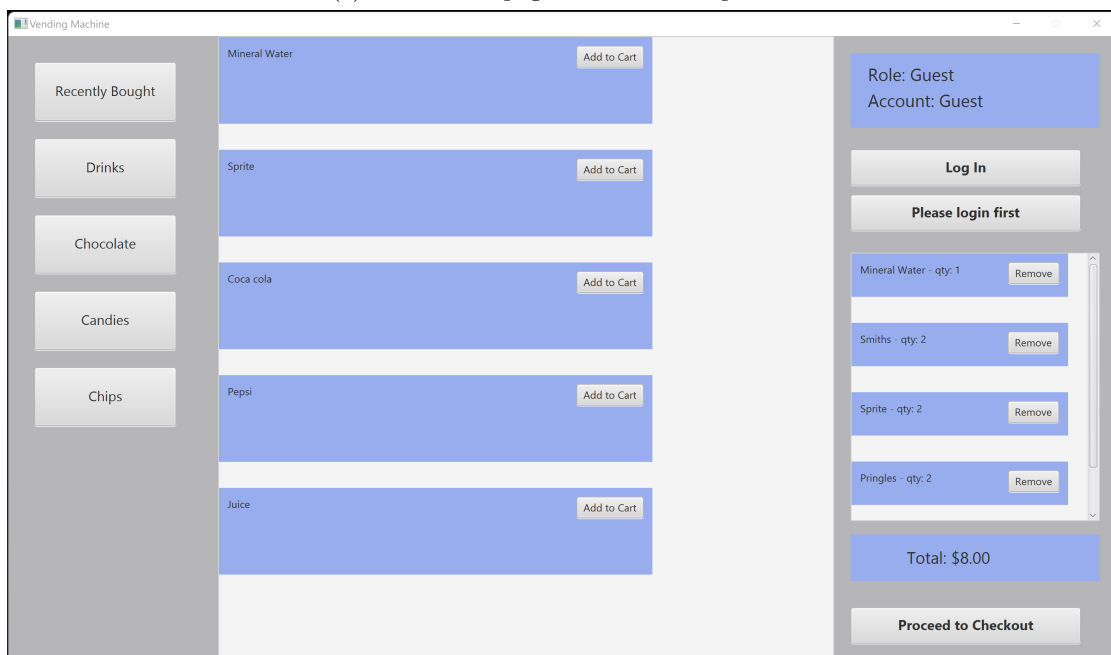
The exact user stories which were planned for this sprint are listed below. These can also be seen in Figure 2.

User stories for Sprint 2

- As a guest, I want to be able to register and that information is retained and persist in some storage so that I can become a registered user.
- Basic UML Diagram
- As a user paying with card, I want to be able to input my card details so that I can pay for my order
- As a user, I want to be able to see the available categories in the vending machine along with their respective items, so that I can add them to a cart
- As a Cashier, I want to be able to access my Cashier’s portal, so I can perform my Cashier duties



(a) The default page at the end of Sprint 1



(b) The new and improved default page, as per the clients request

Figure 1: A comparison of the old and new layout for the default page

- As an Owner, I want to be able to access my Owner's portal, so I can perform my owner duties
- As a Seller, I want to be able to access my Seller's portal, so I can perform my seller duties
- As a user, I want to be able to check-out so I can choose whether to pay with cash or card
- As a user, I want to be able to log in or log out from the home page so that I can change roles
- As a user paying with cash, I want to be able to input a specific amount of currency so that I can pay for my order
- As an owner, I want to be able to manage the roles of all the users of the vending machine so that I can allow users to become sellers, cashiers, or owners (Client Request)
- As an owner, I want to be able to produce a report summarising the list of all vending machine users and their roles
- A mid-fi diagram of how the whole transaction and managing users should look
- As an owner, I want to be able to see all the users for a particular role so that I can keep a track of how many users have special access to the vending machine
- As an owner, I want to remove a user from the vending machine application, so that I can manage when a user is inappropriately using the vending machine
- Rotate the tabs on the Default Page so it is horizontal, and if possible make it larger (Client Request)
- As an owner, I want to be able to view all of my transactions in a tabular (CSV) format, and for all past transactions to persist until deleted.
- As an owner, I want to be able to see which users made with pasts transactions and to be able to easily extract said data into payment collection software

Issue ID	User Story	Category	Status
SFA-87	As a cashier, I want to be able to view and update the amount of each denomination in the vending machine so that customers can pay with cash		TO DO
SFA-86	As a user paying with card, I want to be able to save my card details so that next time I can order products faster		TO DO
SFA-17	Create GUI Mock ups for the rest of the program	ADMIN	TO DO
SFA-40	As a customer (guest or registered), I want to be able to add items to my cart and view the total price of the items in the cart, so that I may buy more than 1 item at a time.	TRANSACTION	TO DO
SFA-49	As a registered customer, I want to be able to see 2 buttons once I have started the transaction (cash or credit), so that I can choose how I want to pay.	TRANSACTION	TO DO
SFA-50	As a customer (guest or registered) once I have finish adding to the cart I want to be able to go to the checkout page.	TRANSACTION	TO DO
SFA-51	As a customer (guest or registered) I want to be able to see a summary of my cart, it's items, and its total price at the checkout page.	TRANSACTION	TO DO
SFA-52	As a registered customer, I want to be able to fill in my card details and save it, so I can pay for my items and it will be quicker the next time around for me to pay.	TRANSACTION	TO DO
SFA-53	As a customer (guest or registered) , I want to be able to manually enter and pick the cash amount I want to put into the machine so that I can pay for my items with cash.	TRANSACTION	TO DO
SFA-54	As a guest user, I want to be able to fill in my card details to pay for my bill.	TRANSACTION	TO DO
SFA-48	As a new customer (guest or registered) I want the previous customer's cart to be cleared and logged out should they have logged in.	UI	TO DO
SFA-15	High Fidelity GUI Mockups	ADMIN	TO DO
SFA-55	As an Owner, I want to add/remove Seller or Cashier or Owner user(s), so that I can manage all my employees roles.		TO DO
SFA-56	As an Owner, I want to modify the items details and the available changes in the vending machine), so that I make sure there is enough change for customers.		TO DO
SFA-57	As an Owner, I want to see a list of usernames in the vending machine with the associated role, so that I can manage all roles.		TO DO
SFA-58	As an Owner, I want to see a summary of cancelled transactions, so that I can see why customers are cancelling.		TO DO
SFA-62	As a Owner, I want to be able to Log out , so that the next person cannot access my account.	AUTHENTICATION	TO DO
SFA-63	As a Seller, I want to be able to Log out, so that the next person cannot access my account.	AUTHENTICATION	TO DO

Figure 2: User story backlog from the start of Sprint 2

2.2 User Ideas to Propose to Client

As well as the features outlined by the client both in the project specifications and at the end of Sprint 1, the team decided to propose, implement and test other features which would be presented to the client at the end of the sprint. These ideas were as follows:

User Ideas to Propose to Client

- As an owner, I want to be able to view all of my transactions in a tabular (CSV) format, and for all past transactions to persist until deleted.
- As an owner, I want to be able to see which users made with pasts transactions and to be able to easily extract said data into payment collection software

Both of these ideas were successfully implemented and tested by the end of the sprint. Upon presentation to the client, he said he was satisfied with the inclusion of these features.

2.3 Mid-fi Diagrams

To complete the set user stories, mid-fi diagrams of all the pages that should've been completed during this sprint were drawn by the product owner, and posted on the Jira project board (see Appendix B. This was done to convey how the project owner wants each page to roughly look and how they should function.

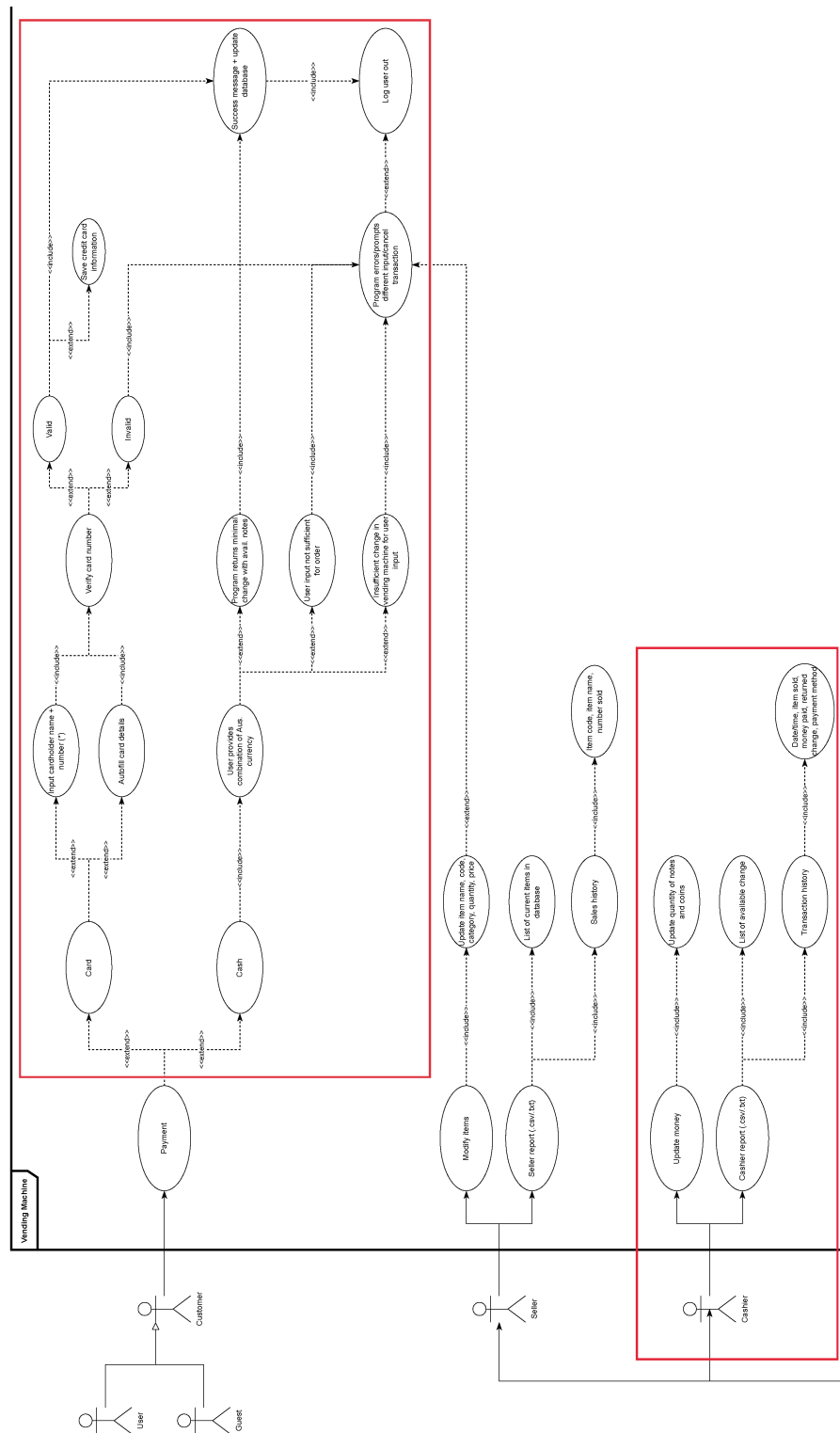


Figure 3: The selected portions of the use case diagram this sprint focused on

2.4 Product and Sprint Task Boards

Like last sprint, the creation and tracking of issues was managed using [Jira Software](#). However, this sprint there were a few key changes that were implemented in accordance with the Sprint Review from sprint one.

Firstly, the implementation of priorities with (user) stories. This sprint we assignment issues/stories based on a combination of their relative priorities and the assigned story points.

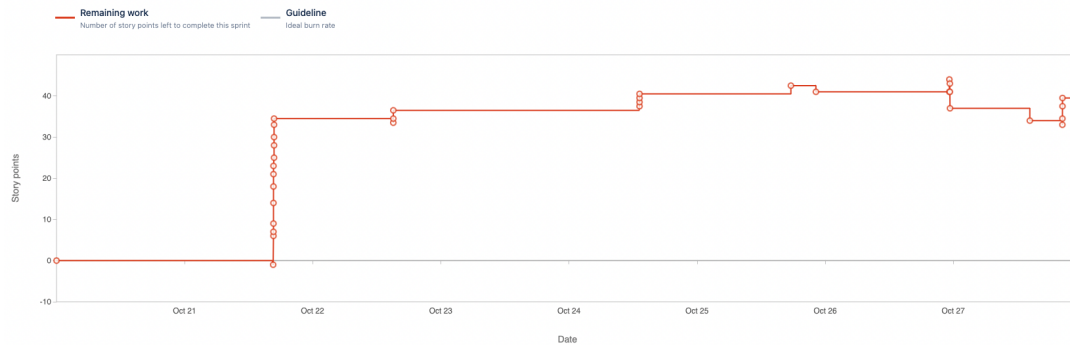


Figure 4: Sprint 2 burndown chart

2.5 Artifacts

2.5.1 Standups

Unlike Sprint 1, during Sprint 2, due to all team members being really busy with other assignments, the team only managed to have 4 stand ups. All of these stand ups went for about 20 minutes, during which each team member got to report their progress and voice any concerns they had. All stand ups were transcribed in the meeting minutes page, which is located under project pages in Jira. The meeting notes can also be found in the appendix of this report

2.5.2 Sprint Review

For the sprint review the team discussed the overall experience that each of the team members faced. Each member assessed their performance by comparing the objectives that were set at the beginning of the sprint to the results at the end of the Sprint. This was overseen by the Product Owner and the Scrum Master. Key points noted during the review were, team members did most of the development towards the end, and hence next sprint should be started earlier. Formatting and Documentation were not up to standard and should be considered more carefully in the next sprint. However, the team also appreciated the noticeable improvement in performance from the previous sprint. In the previous sprint the allocated number of story points were 20 (approx), and this sprint there were 40 points (approx.) worth of user stories. However, developers managed to

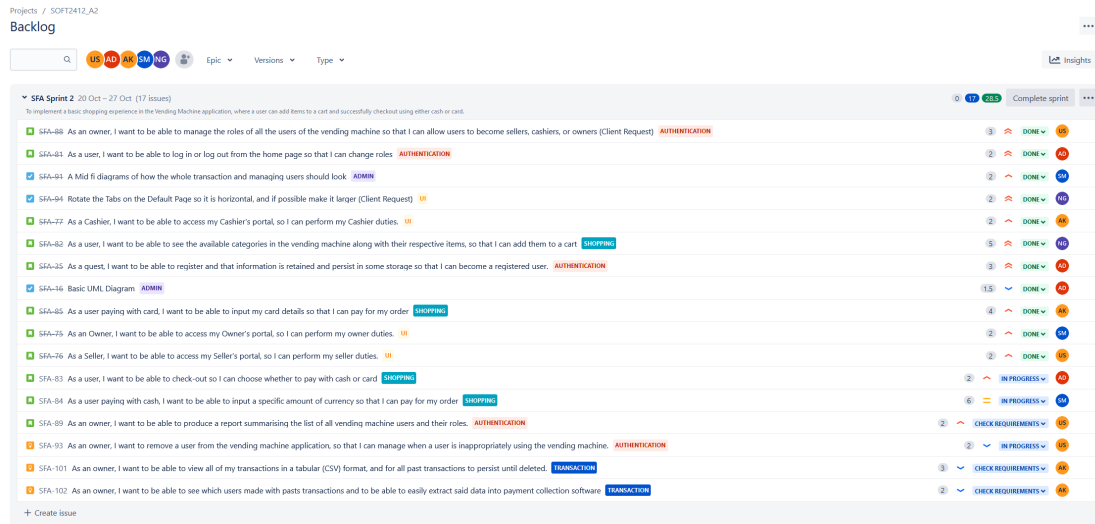


Figure 5: Final Backlog for Sprint 2

almost complete all allocated user stories. Currently, almost all user stories, have been completed or are pending for a final check before the stories can be moved to complete.

3 Agile Development Practices

The team continued to use staple agile tools such as Jira, Git, and Gradle to ensure rapid and smooth development of the software. All code developed during the sprint have been pushed and saved to a remote git repository, and all other artefacts have been upload to Jira.

3.1 Git

A private repository *vending-machine* was created in the University of Sydney's GitHub enterprise under the SOFT2412-2022 organisation. All 5 members had access to the repository and used it to allow collaboration between team members and maintain the project source code.

3.1.1 Branches

There were some key differences employed in the agile practices of this project compared to our previous project *Currency Converter*. In Project 1, each team member was essentially assigned management of one part of the application; for example, member 1 would be assigned to the creation and management of the database, while member 2 would be assigned to developing the GUI. In this method of development (commonly referred to as *silo-driven development*), each member wanted to focus on their own code without worrying about integrating their design with the rest of the software until required. Hence, we used up to 6 separate git branches to produce each major working part of the app.

The process intrinsically led to large features being developed locally and merged to the main branch all at once. This is not in the spirit of the agile process, nor is this facilitating a communicative team environment.

Hence, in this project each member would work on functionalities across the entire app to follow the agile methodology. Thus the team chose to work solely on the main branch.

3.2 Testing

3.2.1 Incremental Test Case Design

Tests were completed by all team members incrementally as features were added to the main application. This ensured that all components were covered as they were implemented and defined the scope and limitations of each function, allowing for team members to further refine these methods and develop more inclusive, robust algorithms. For example, when the card payment feature was tested, the incremental addition of test cases allowed for the respective developer to consider all possible input types (e.g. string or float inputs, with or without spaces, etc.) and ensure full inclusivity for all possible values that could be entered into the input fields.

In terms of how this was done, developers would refer extensively to the user stories on the Jira board and would inspect the relevant acceptance criteria for each story. Once all elements of the acceptance criteria were fulfilled, tests were made to verify each criterion and the respective files were adjusted in the case that any tests were failed. The method used was the standard consideration of all possible values vs all considered values: test cases were developed intuitively depending on what developers would expect of worst case scenarios, and if the tests were failed for said scenarios then the corresponding functions were adjusted until all test cases were passed.

3.2.2 Integrated Component Testing

Another major aspect of testing that was considered by all developers was the acceptance tests for integrated components. As the application is intended to be robust and cohesive, all functions needed to work when used together. In order to ensure that all components successfully worked together, extensive test cases were created that test the use of multiple components simultaneously (in conjunction with the individual tests for each component) to simulate real use-case scenarios. For example, a test case was developed where a new database was created to simulate a new instantiation of the entire app as a whole, and where various functions were called on entry values to test inclusivity of possible alternate data entries into the tables.

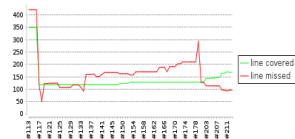
3.2.3 Iterative Testing

Furthermore, when testing some minor algorithms such as those used in validating card payments, iterative testing methods were used to ensure that an assortment of values were considered during testing. This process includes creating a list of a range of values and implementing a loop that asserts the expected vs true output of the algorithm for each value. This method was successful as it streamlined the process of value selection for minor test cases.

3.2.4 Test Coverage

An average of 76% test coverage was maintained throughout the entire sprint (see Figure 6).

Package: VendingMachine



Coverage Summary

name	instruction	branch	complexity	line	method	class
VendingMachine	M: 308 C: 860 74%	M: 7 C: 31 82%	M: 32 C: 43 57%	M: 96 C: 168 64%	M: 27 C: 29 52%	M: 1 C: 3 75%

Coverage Breakdown by Source File

name	instruction	branch	complexity	line	method	class
Database	M: 145 C: 716 83%	M: 2 C: 24 92%	M: 3 C: 26 90%	M: 42 C: 128 75%	M: 1 C: 15 94%	M: 0 C: 1 100%
SceneManager	M: 106 C: 0 0%	M: 0 C: 0 100%	M: 16 C: 0 0%	M: 31 C: 0 0%	M: 16 C: 0 0%	M: 1 C: 0 0%
Session	M: 57 C: 61 52%	M: 4 C: 2 33%	M: 12 C: 9 43%	M: 23 C: 23 50%	M: 10 C: 8 44%	M: 0 C: 1 100%
Transaction	M: 0 C: 83 100%	M: 1 C: 5 83%	M: 1 C: 8 89%	M: 0 C: 17 100%	M: 0 C: 6 100%	M: 0 C: 1 100%

Figure 6: Testing coverage report at the end of Sprint 2

app

app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
VendingMachine	308 of 1,168	73%	7 of 38	81%	32	75	27	1
Total	308 of 1,168	73%	7 of 38	81%	32	75	27	1

Figure 7: Jacoco Test coverage for report at the end of Sprint 2

3.3 Gradle

3.3.1 Build Automation

Just like in Sprint 1, Gradle was used extensively to help with build automation and testing. The agile manifesto and Gradle Build automation was implemented using various different Gradle tasks and dependencies. For example, the team defined custom tasks like fatJar, shadowJar, jtr and Javadoc to be able to document, compile and report on the code. This allowed the team to not only quickly build and run the program, but also quickly document the code as a Javadoc and compile the code into a jar file. Lastly, we used Gradle in combination with a Makefile to quickly push to GitHub and clean the build as necessary.

3.3.2 OpenCSV

To allow for CSV files to be easily accessible, readable and writable, the OpenCSV Java library plugin was added to the build.gradle file. This streamlined the process of recording and storing transactions between different instances of the app and created a concise and easily legible and exportable transaction history file for the owner.

3.4 CI/CD Tools and Practices

In Sprint 0 and Sprint 1 the CI/CD Pipeline the team focused on creating and refining the Continuous Integration and Continuous Development Pipeline. The first notable change from the first assignment has been to switch from multiple feature branches to just using the main branch (see section 3.1 for more on git). This, of course, has significantly increased the number of pushes to the main branch on GitHub, thereby increasing the number of times the application was *deployed*.

Gradle, Git and GitHub, Jira, Jenkins and Discord have been used for the CI/CD pipeline. With Gradle, the team's developers are able to automate programming, building compilation, and execution using Gradle commands. Gradle was used with JUnit to create unit and acceptance tests that were pushed to Git and GitHub.

Git and GitHub were used to collaborate on the application source code, and manage versioning of our program. [Our Jira board](#) was used to keep track of progress of the current sprint and was used to keep a track of issues.

[Jenkins](#) was used to automate building, compiling and testing the code before deployment. One limitation of the program and the way that the team incorporated testing was that there was no end-to-end testing. This is due to the nature of the program, and how it has a very complex GUI structure. This raises two vulnerabilities with the current application: a lack of testing the GUI, and end-to-end testing. The team has made up for this lack of code testing by testing the application on various edge cases. Jenkins was also used to cross reference test results, and testing coverage with that of the results on the local computers of the developers. This ensured that testing coverage on all devices was always at least 75%.

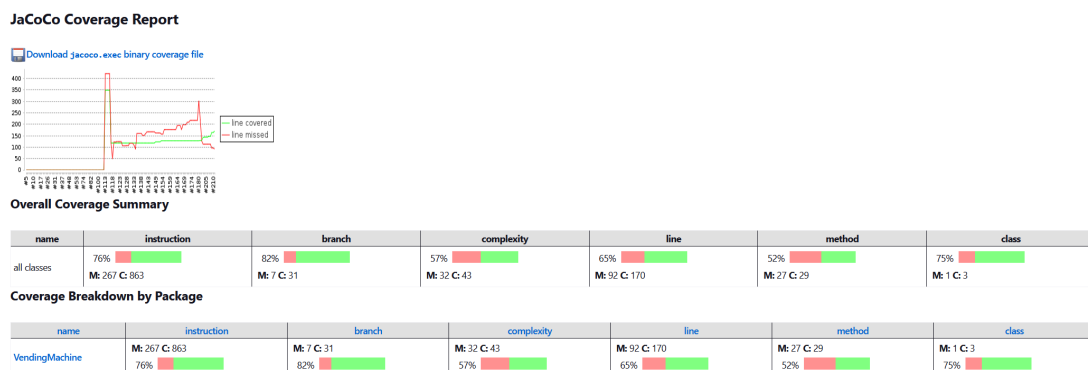


Figure 8: Jenkins recording test coverage

Lastly Discord and the Jenkins Discord API was used as a tool for the final part of the Jenkins Pipeline. Discord broadcast to the team when a build failed, and from which commit. This created a feedback loop informing the team of whether the build was successfully compiled or not. It also gave the members of the team access to the recorded artefacts based on particular

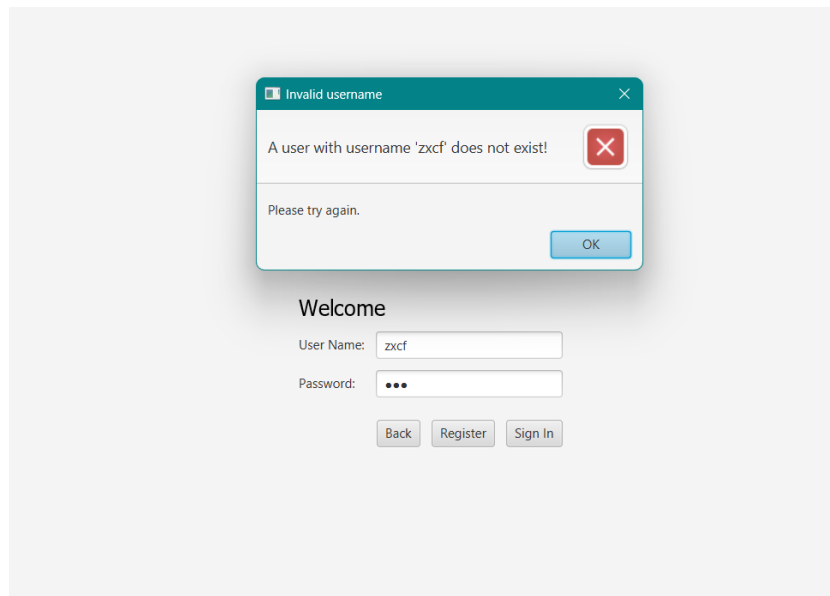


Figure 9: Example of app testing for an edge case

build. This way there was a good system in place where the team could check the artefacts of an exact build (based on build failure or success for example) with just one link (see Figure 10).

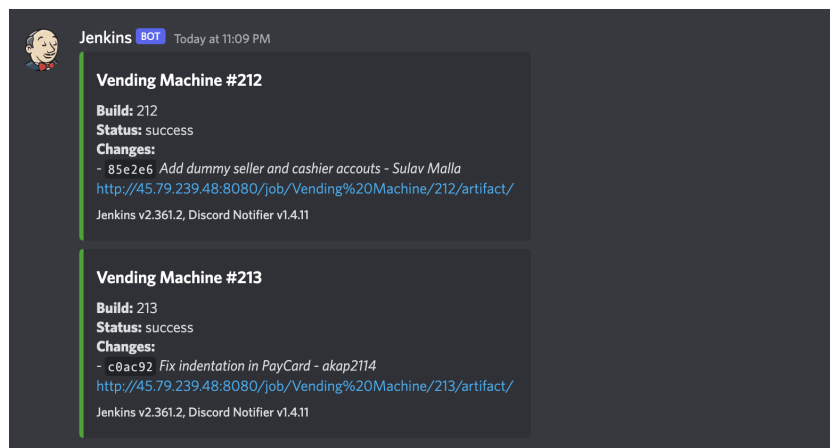


Figure 10: An example of how the Discord webhook was used

4 Application Development

4.1 Group Collaboration

In Sprint 2, Group Collaboration was reasonably good, we had stand-ups every couple of days, which was less frequent than in the last sprint. GitHub, Jira, and Jenkins were all still excellent tools for version control, agile practices and build automation.

The Individual contribution was not as even as expected this sprint as some members were busier than others with other assignments. Nemo implemented the default page design, rotated the tabs and added the add-to-cart feature. Ankit finished the credit card payment system. Udit added more functionality to the database. Antriksh added the checkout page and did other general coding and testing. Sulav made the proceed to portal button go to the correct portal and did other general coding and testing.

4.2 Application Design

The following is an overview of the Vending Machine Application Front-End design.

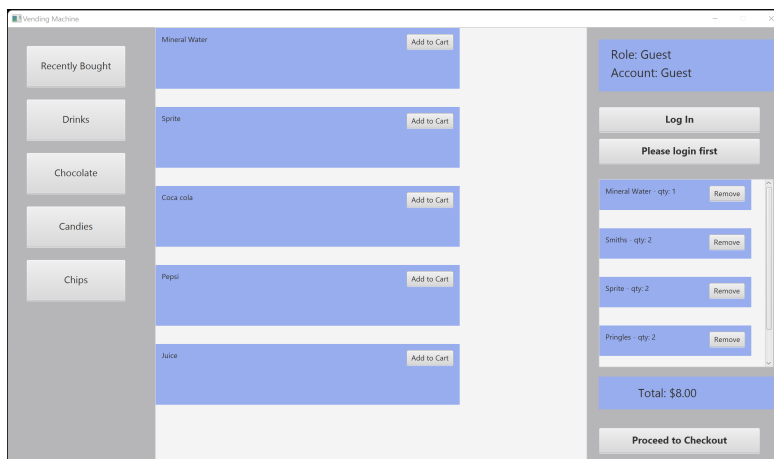


Figure 11: Default Page using FXML

A new tool used in sprint 2 was FXML. FXML was used to create the default page shown in Figure 11. Furthermore, a DefaultPageController class was created and linked to this FXML file to write the logic for all the buttons on the Page.

As shown in Figure 12, SceneBuilder was used drag, resize and add properties to elements and then this could be saved as an FXML file and read by the FXMLloader. SceneBuilder also displays the Document Object Model hierarchy on the left, making it easier to see the parent and child objects in the Page. To edit the functionality of a JavaFX button in FXML, in SceneBuilder the on action value should be changed to the name of the function in the linked controller class.

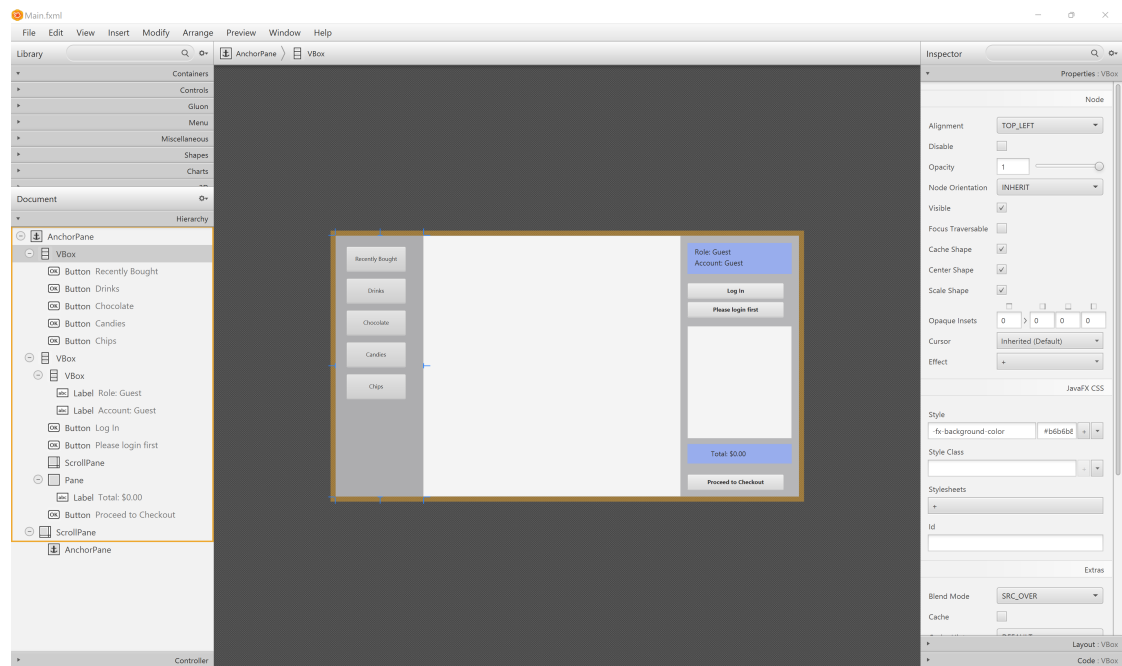


Figure 12: Scene Builder used to edit FXML

A UML diagram

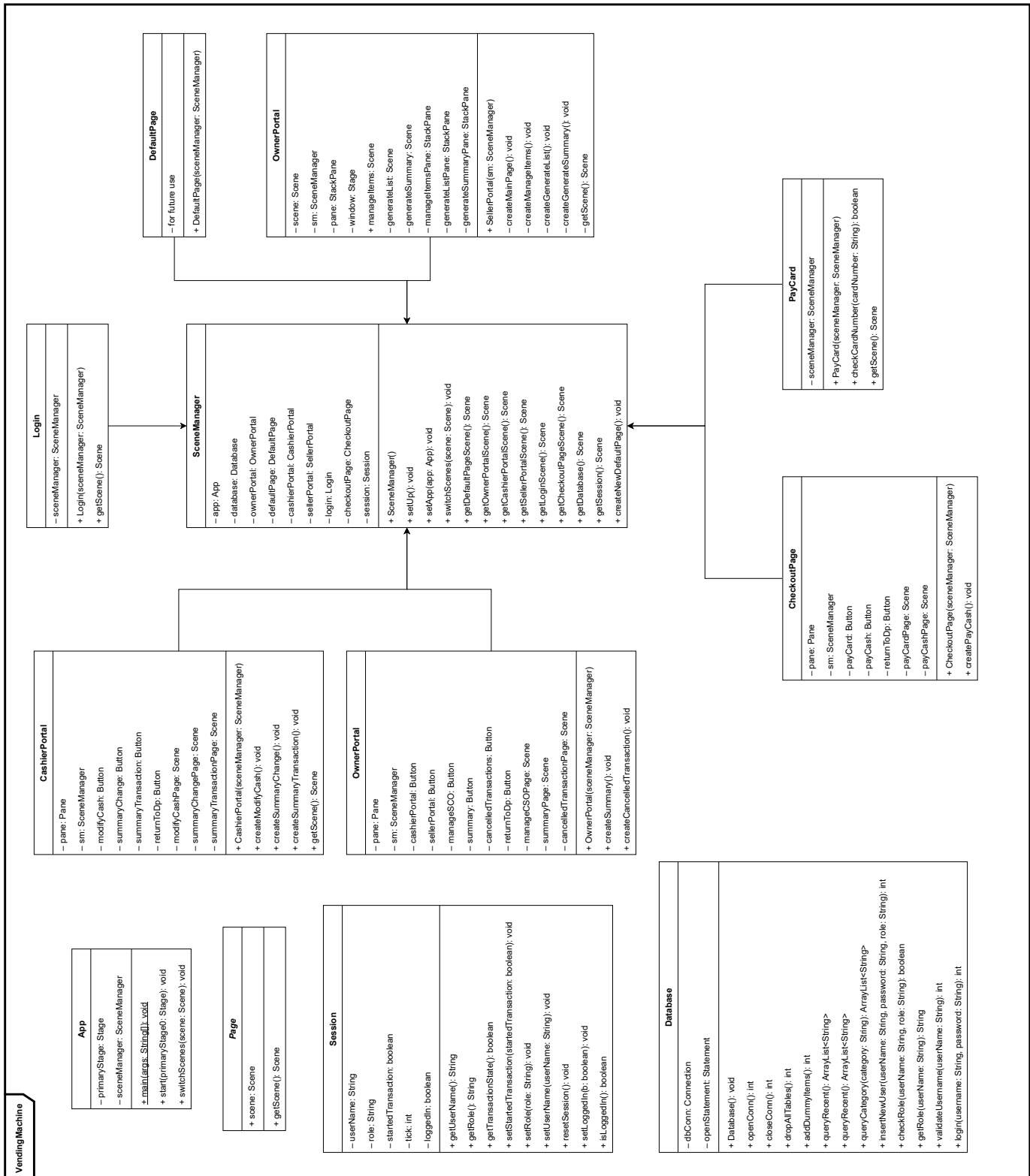


Figure 13: The current latest UML diagram for the application

B Mid-fi diagrams

B.1 Transaction pages

Checkout

Item	Quantity
Coke	3
Fanta	5
.....	
.....	
.....	

Total: \$7.50

Pay by Cash

Pay with Card

Cancel

CardHolder's Name

Credit Card Number

Pay

Cancel

Option 1 of cash payment

Total

\$ 15.50

Due

\$ 6.50

Enter Amount

6.50

Pay

Cancel

Option 2

\$ 100

\$ 2

\$ 50

\$ 1

\$ 20

¢ 50

\$ 10

¢ 20

\$ 5

¢ 10

¢ 5

Total

\$ 15.50

Due

\$ 6.50

Change

\$ 0.00

Cancel

B.2 Manage users page

Manage Users

Username

Password

Select Role

C Meeting Minutes

Meeting minutes

24/10/22 The meeting started at 11.00 PM.

Written By - Sulav

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- Udit came up with 2 ideas that can be presented to the client.
- Udit started working on the Owner Page.
- Udit did the Executive Summary for the sprint report.
- Ankit accidentally completed some of Antriksh's assigned work and has made the check-out page.
- Ankit has made pay by card page, and has done some of the testing for it.
- Antriksh finished the UML for the sprint.
- Antriksh has started on the Register page.
- Nemo has planned how to make the cart work.
- Sulav done with the all the Mid fi diagrams for the sprint.
- Sulav has refactored the Jira backlog and it is now cleaner.
- Sulav has started on cash page, implemented all the buttons.

25/10/22

The meeting started at 11.00 PM.

Written By - Nemo

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- Udit has made a scaffold of his tasks.
- Sulav didn't manage get time to do tasks.
- Ankit has finished all of his assigned work and so decided to refine other aspects of the program.
- Ankit proposed some user ideas to implement into the application.
- Antriksh didn't manage get time to do tasks, he is weeping.

- Nemo has to finish displaying items and adding to cart.

26/10/22 The meeting started at 9.00 PM.

Written By - Ankit

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- Team members discussed which elements had been completed and which elements were left to implement for Sprint 2.
- Ankit completed testing for the PayCard page and its methods and continued refining error messages and alerts for all pages.
- Ankit discussed and implemented the user ideas he proposed in the last meeting.
- Antriksh finished the UML for the sprint.
- Antriksh has started on the Register page.
- Nemo finalised development on the cart UI.
- Nemo cleaned up the SRC directory into various subdirectories for clarity.

27/10/22 The meeting started at 10.00 PM.

Written By - Ankit

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- All team members finalised the sprint report and consolidated their contributions in the documentation for Sprint 2.
- Udit implemented an urgency hierarchy system for the Jira page to supplement the story point value system.